

이력사항



장현우 1999.04.17
백엔드 개발자

- 010-9378-6763
- gusdn6763@gmail.com
- 서울특별시 구로구 구로동
- github.com/gusdn6763
- solved.ac/profile/gusdn6763

Spring 기반 백엔드, AWS 클라우드 인프라 구축,
데이터베이스 설계 경험을 보유하고 있습니다.

자격증

2026.06	네트워크관리사 2급
2025.06	정보처리산업기사
2019.06	컴퓨터활용능력 2급

학력

2023.02~2026.02	한국방송통신대학교 컴퓨터과학과
2015.02~2018.02	선린인터넷고등학교

경력

2021.06~2024.01	한국가상현실 솔루션개발 2팀
-----------------	-----------------

활동/교육

2026.03~2026.10	Spring 백엔드 부트캠프 (코드잇)
2020.02~2021.03	42 서울 1기
2019.06~2020.01	유니티를 활용한 게임프로그래밍 (조은컴퓨터아카데미)

기술 스택

백엔드

Java, Spring (Framework, Boot, Batch, Security), JPA, QueryDSL, OAuth2, JWT, PostgreSQL

인프라

AWS (ECS, RDS, S3, CloudWatch, Secrets ManageR, Route 53, ALB, ACM)

개발 도구

IntelliJ, GitHub, Docker, TortoiseSVN, Visual Studio, VSCode

프로젝트 목록

세 개의 주요 프로젝트를 통해 백엔드 시스템 설계, 배치 처리, 프로그래밍 개발 경험을 보유했습니다.



덕후감

도서 이미지 OCR 및 ISBN 매칭 서비스



개발 인원

5명 (팀 프로젝트, 팀장)



개발 기간

2026.06.22 ~ 2026.07.13



HR-Bank

Scheduler로 데이터를 관리하는 Open EMS



개발 인원

5명 (팀 프로젝트, 팀장)



개발 기간

2026.05.14 ~ 2026.05.26



게임/그래픽 프로젝트

VR 환경 게임 개발
다양한 게임 프로그래밍 패턴 적용



개발 인원

1명 (개인 프로젝트)



개발 기간

2019.06~2021.06

Project.1

덕후감

도서 이미지 OCR 및 ISBN 매칭 서비스

01

목차

1. 개요
2. 기획
3. 설계
4. 기여한 부분 (3)

개요

덕후감

도서 이미지 OCR로 ISBN을 추출하고 Naver API로 도서 정보를 자동 등록하는 도서 리뷰 커뮤니티입니다.

팀장을 맡았으며 프로젝트 관리·기초 구축, 인증/인가(JWT·OAuth2), 랭킹 집계 배치 서버, CI/CD, AWS ECS 배포를 담당했습니다.



기획

① 협업 방식

03 분석 DEOK-HUGAM

Notion

프로젝트 문서화

- 요구사항, DB Schema, API 명세 관리
- 회의록 및 트러블슈팅 기록

중급 프로젝트

Jira

이슈 및 일정 관리

- 스프린트 단위 백로그 생성 및 담당자 지정
- 칸반 보드를 통한 작업 상태 실시간 추적

GitHub

코드 형상 관리

- Feature Branch 전략
- PR 기반 코드 병합 및 변경 이력 관리

Notion 문서화 → Jira 작업 관리 → GitHub 코드 관리 → Dev Merge 코드 통합

② API 명세 요약

03 분석 DEOK-HUGAM

사용자 관리 회원/인증

회원가입	POST	/api/users
로그인	POST	/api/users/login
사용자 정보 조회	GET	/api/users/{userId}
사용자 정보 수정	PATCH	/api/users/{userId}
사용자 논리 삭제	DELETE	/api/users/{userId}
사용자 물리 삭제	DELETE	/api/users/{userId}/hard
도시 목록 조회	GET	/api/books

리뷰 관리 리뷰/좋아요

리뷰 목록 조회	GET	/api/reviews
리뷰 등록	POST	/api/reviews
리뷰 좋아요	POST	/api/reviews/{reviewId}/like
리뷰 상세 정보 조회	GET	/api/reviews/{reviewId}
리뷰 수정	PATCH	/api/reviews/{reviewId}
리뷰 논리 삭제	DELETE	/api/reviews/{reviewId}
리뷰 물리 삭제	DELETE	/api/reviews/{reviewId}/hard
리뷰 댓글 목록 조회	GET	/api/comments

댓글 관리 댓글

리뷰 댓글 목록 조회	GET	/api/comments
댓글 등록	POST	/api/comments
댓글 상세 정보 조회	GET	/api/comments/{commentId}
댓글 수정	PATCH	/api/comments/{commentId}
댓글 논리 삭제	DELETE	/api/comments/{commentId}
댓글 물리 삭제	DELETE	/api/comments/{commentId}/hard

알림 관리 알림

알림 목록 조회	GET	/api/notifications
알림 읽음 상태 업데이트	PATCH	/api/notifications/{notificationId}
모든 알림 읽음 처리	PATCH	/api/notifications/read-all

대시보드 관리 랭킹

인기 도서 목록 조회	GET	/api/books/popular
인기 리뷰 목록 조회	GET	/api/reviews/popular
피워 유저 목록 조회	GET	/api/users/power

도서 관리 도서/ISBN

도서 목록 조회	GET	/api/books
도서 등록	POST	/api/books
OCR 기반 ISBN 인식	POST	/api/books/isbn/ocr
도서 상세 정보 조회	GET	/api/books/{bookId}
도서 정보 수정	PATCH	/api/books/{bookId}
도서 논리 삭제	DELETE	/api/books/{bookId}
ISBN으로 도서 정보 조회	GET	/api/books/info
도서 물리 삭제	DELETE	/api/books/{bookId}/hard

공통 API 규칙

구분	내용
인증 방식	헤더(백세스 토큰) 쿠키(리프레시 토큰)
인증 사용자 식별	AuthenticationPrincipal 어노테이션
페이지네이션	cursor after limit
정렬 방향	DESC ASC
여러 응답	상태코드 + 헤더지 반환

③

프로젝트 - 덕후갑

7월 13일 만일 +1명 88 공유

14 15 16 17 18 19 20

21 22 23 24 25 26 27

회의 문서화 작업

28 29 30 7월 1일 2 3 4

개발 기간 배포 기간 + a

요청 마감 기한

문서

- API 명세서
- ERD
- 트러블 슈팅
- 메카닉

협업 규칙

- git 전략 및 협업 규칙

▼ 개발 리스트 & 담당자

새 데이터베이스

As 제목	담당자	상태
API 명세서 작성	우진 정	완료
ERD 설계	성규 김 전민지	완료
이슈 작업	지 지혜	완료
프로젝트 기초 구축	정현우	완료
인증 / 연가 작업	정현우	완료
		시작 전

① Notion 문서화, Jira 이슈·일정 관리, GitHub 형상 관리로 협업 방식을 운영 했습니다. 브랜치는 main → dev → feature로 구성하고 dev는 항상 실행 가능한 상태로 유지 했습니다. 기능 완료 시 PR로 dev에 병합하였고, 팀원 간 dev 브랜치를 상시 최신화 하여 충돌을 최소화했습니다.

② Notion에 API 명세서를 문서화하여 엔드포인트와 요청·응답 형식, 인증 방식을 팀 전체가 공유할 수 있도록 관리했습니다.

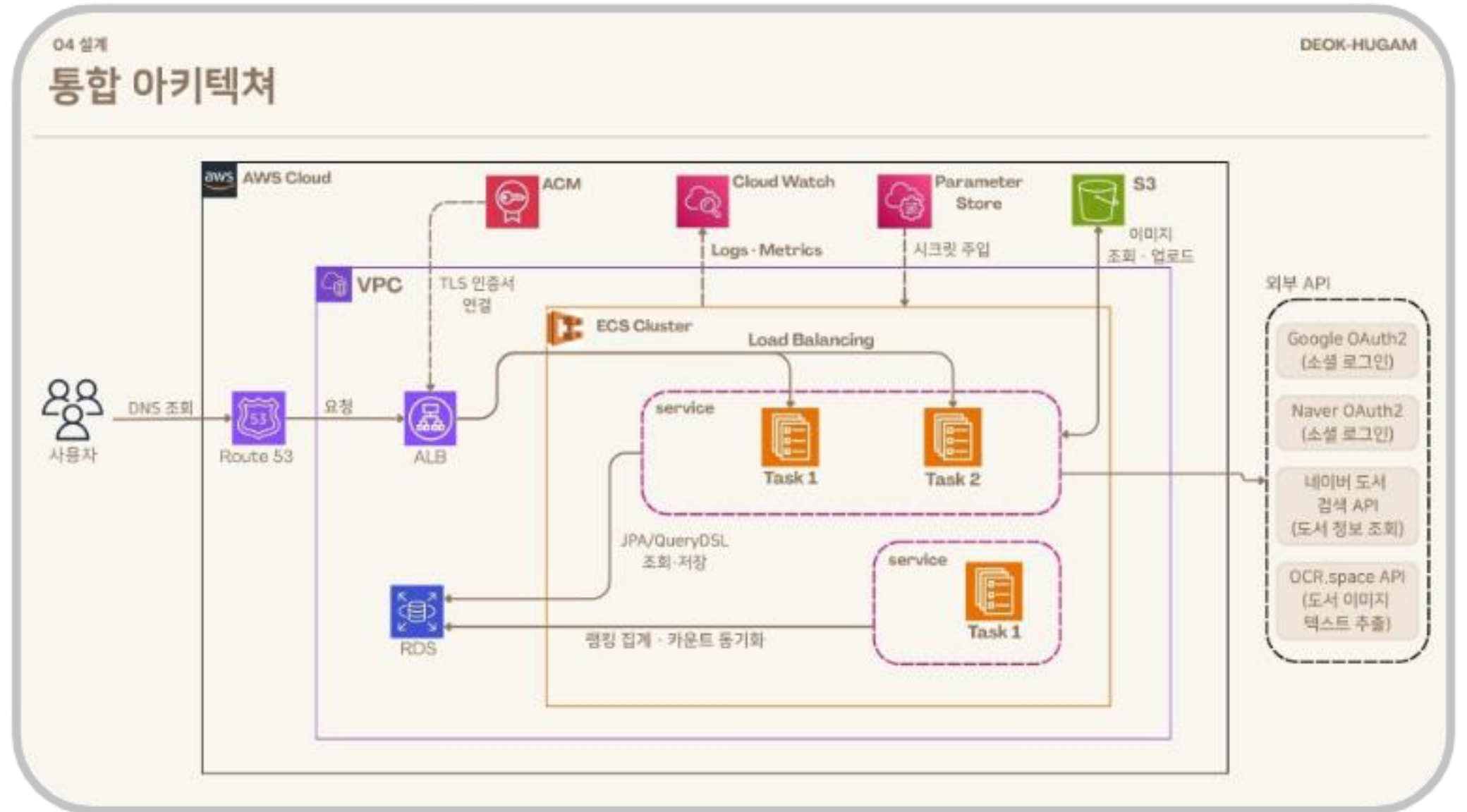
③ 팀장으로서 회의록을 직접 작성하고 일정 조율과 작업 분배를 담당했습니다. 개발 리스트와 담당자를 관리했으며, GitHub 저장소와 페이지 관리를 주도했습니다.

설계

①



②



③



①

사용자·도서·리뷰·댓글·알림 등 핵심 도메인 외에, 조회 시 실시간 집계 부하를 피하기 위해 랭킹 결과 테이블(도서/리뷰/유저)을 분리해 총 11개의 테이블로 설계했습니다.

②

Route 53 → ALB → ECS Cluster로 흐름을 구성하고, 메인 서버는 2개의 Task로 구성해 트래픽을 분산했습니다. 배치 작업이 메인 서버에 영향을 주지 않도록 배치 서버를 별도 서비스로 분리했습니다.

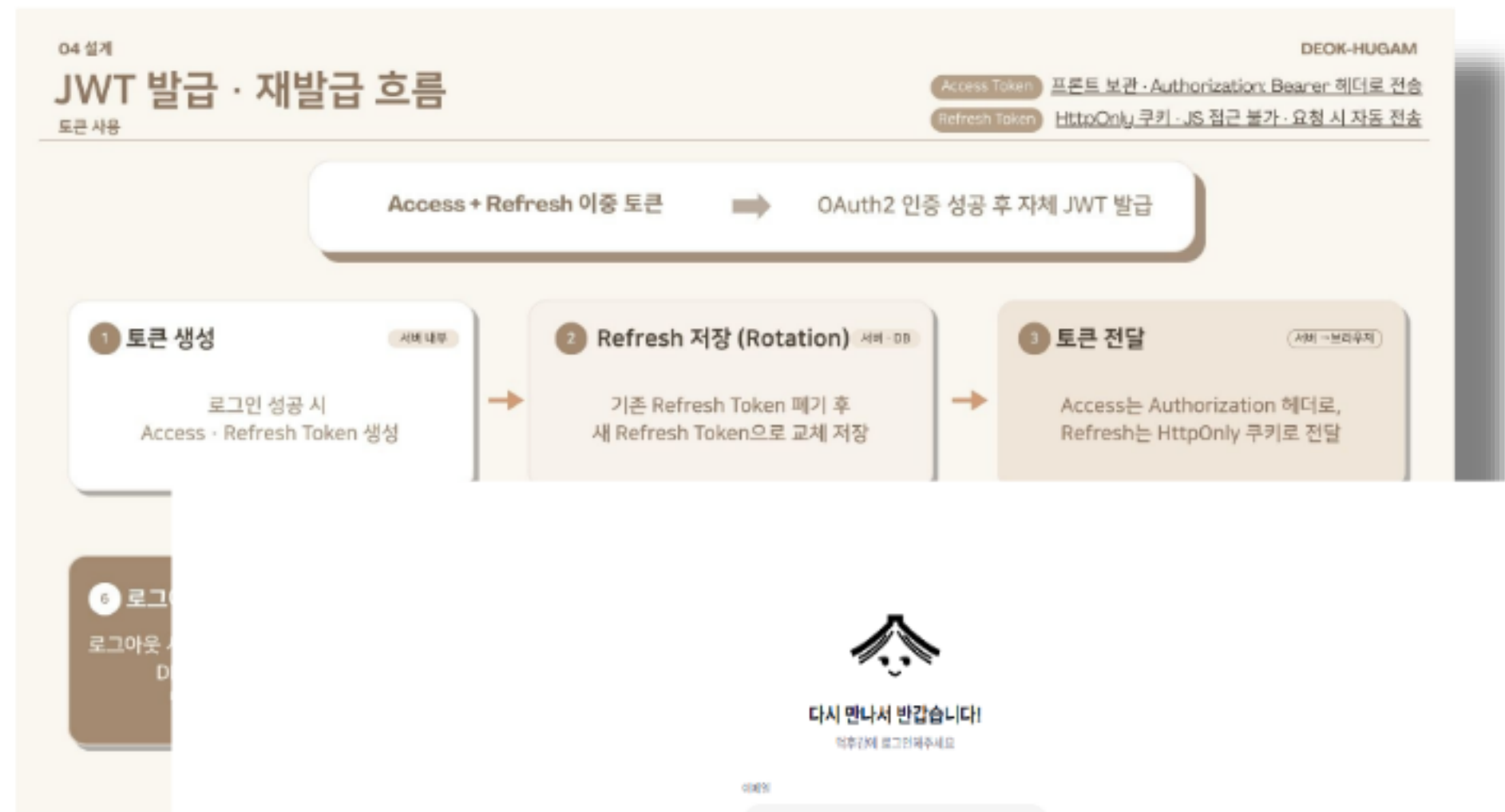
③

다중 서버 확장을 고려해 JWT방식을 채택하고, JPA·QueryDSL, OAuth2, AWS ECS 배포까지 감안해 시스템 아키텍처를 설계했습니다. GitHub Actions로 CI/CD를 자동화했습니다.

기여한 부분 - 1 / 3

인증 / 인가

JWT(Access/Refresh + Rotation) 인증 구조 설계
OAuth2(Naver, Google) 구현



JWT 채택 사유

- ECS 다중 서버 확장성 — ECS 위에서 메인 API 서버가 2개 Task로 수평 확장되는 구조라 추가 인프라와 단일 장애점 없이 서명 검증만으로 인증이 끝나는 JWT를 채택했습니다.
- 로그인 방식 통합 — OAuth2로 신원 확인 후 자체 JWT를 재발급하면, 일반/소셜 로그인의 인증 흐름을 하나로 통일할 수 있기 때문입니다.
- DB 조회 최소화 — 매 요청마다 세션 저장소를 조회해야 하는 세션과 달리, 서명 검증만으로 인증이 끝나 인증 단계의 DB 부하를 줄일 수 있기 때문입니다.

JWT 적용 방식

- Access / Refresh 분리 — Access는 수명을 짧게 잡아 탈취되어도 악용 시간을 제한하고, 재발급용 Refresh를 별도로 두어 사용성을 확보했습니다.
- HttpOnly 쿠키 — Refresh Token을 JS 접근이 차단된 쿠키에 저장해, 가장 위험한 탈취 경로인 XSS를 막았습니다.
- Refresh Rotation — 사용할 때마다 새 토큰으로 교체·폐기하고, 폐기된 토큰이 재사용되면 탈취로 판단해 거부합니다.

기여한 부분 - 2 / 3

보안 설계

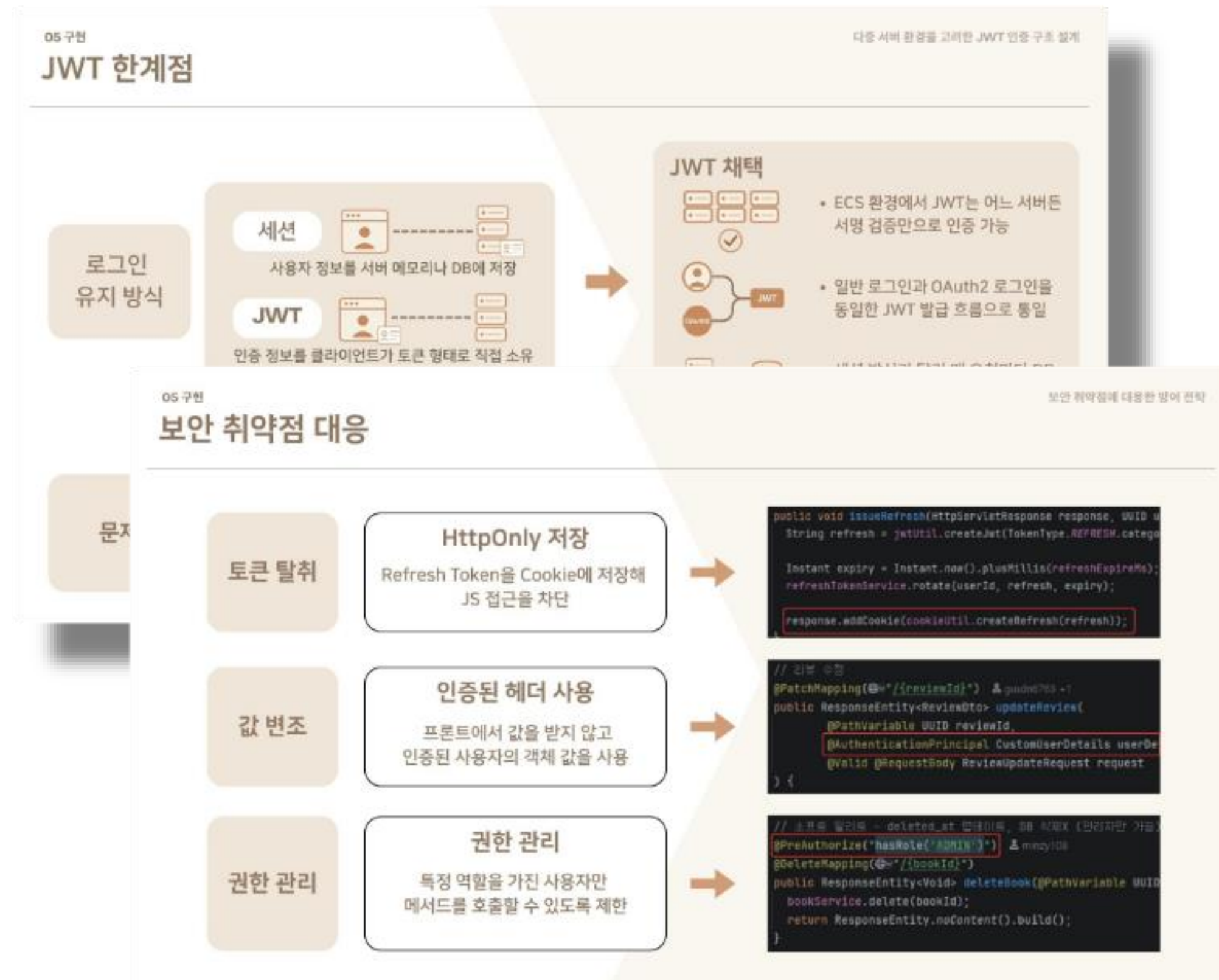
XSS 공격, 요청 값 변조, 일반 사용자의 관리자 API 호출 방어

보안 취약점

- 평문 통신 노출 — HTTP 통신은 암호화되지 않아, 중간자 공격으로 토큰·쿠키가 네트워크 구간에서 탈취될 수 있습니다.
- 권한 없는 리소스 접근 — 로그인만으로는 막을 수 없어, 인증된 사용자가 타인의 리뷰·댓글·도서를 수정/삭제할 수 있습니다.
- 비인가 요청 유입 — 검증 없이 요청이 컨트롤러에 도달하면, 인증되지 않은 사용자가 보호된 API에 접근할 수 있습니다.

보안 해결법

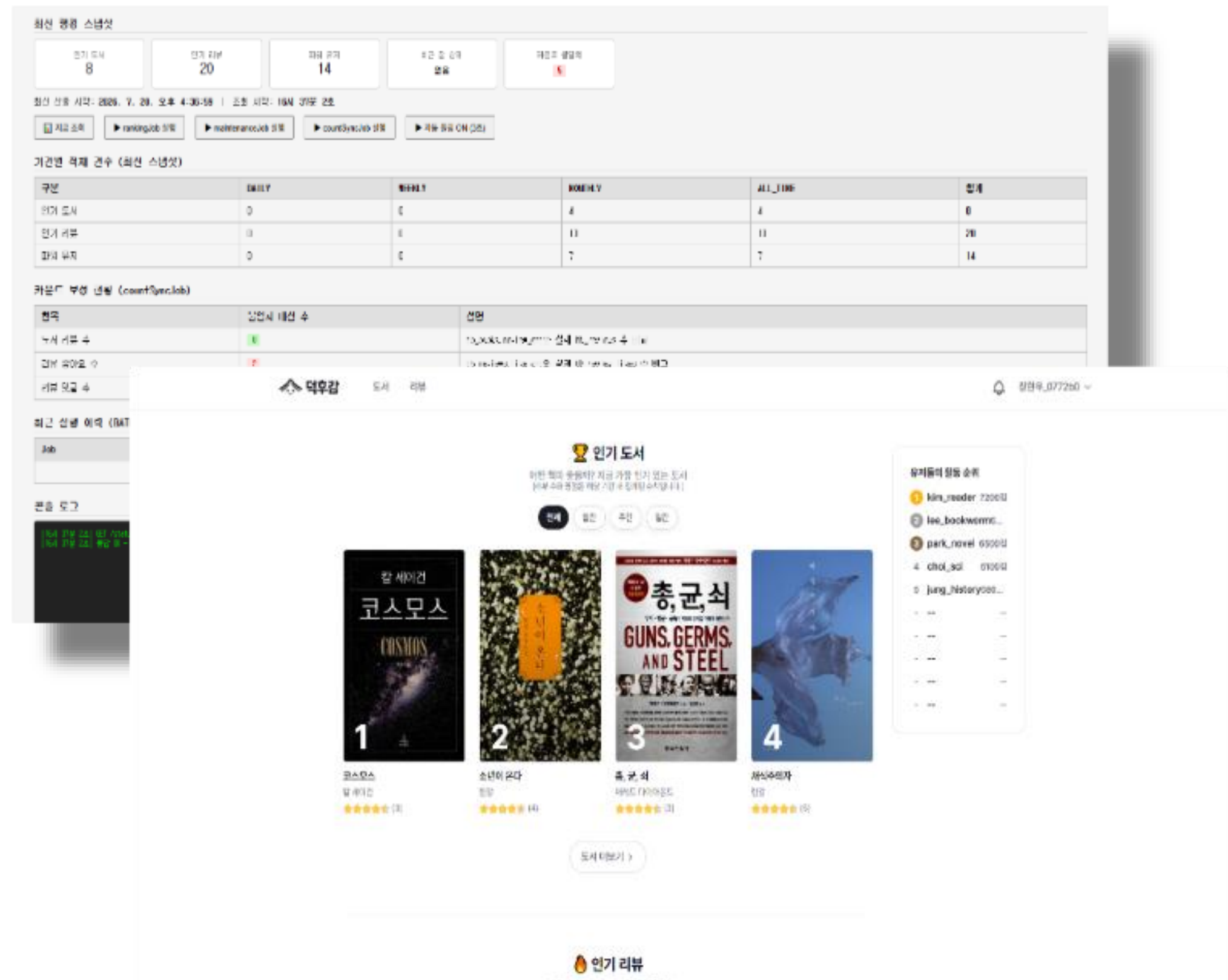
- Https 리다이렉션 — ALB에서 HTTP 요청을 HTTPS로 강제하고 쿠키에 Secure 속성을 적용해, 토큰·쿠키의 평문 전송을 차단했습니다.
- 권한 관리 — 인증된 사용자 ID와 리소스 소유자 ID를 대조해, 리뷰·댓글·도서 수정/삭제는 본인만 수행하도록 제한했습니다.
- 인증된 헤더 — 커스텀 필터가 컨트롤러 도달 전 토큰의 서명·만료를 검증하고, 인증이 불필요한 공개 API는 화이트리스트로 분리했습니다.



기여한 부분 - 3 / 3

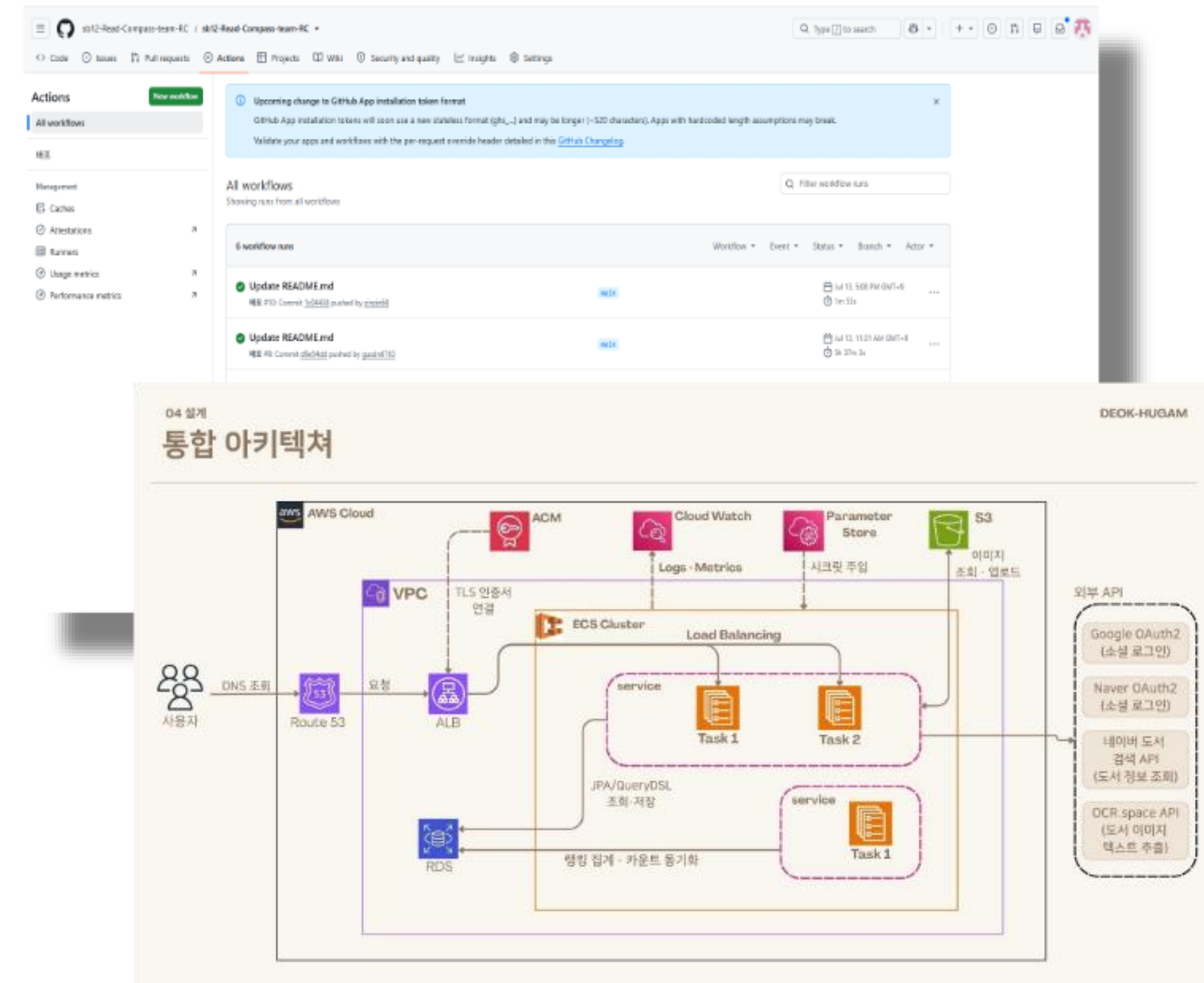
배치 프로젝트 구현

도서·리뷰·유저 랭킹 집계, 알림 정리, 탈퇴 유저 물리 삭제.
배치 프로젝트를 별도 서버로 분리 후 스케줄러로 자동 실행
Actuator 커스텀 메트릭·모니터링 API로 배치 실행 결과 추적



CI / CD 및 아키텍처 배포

GitHub Actions → Docker 빌드 → ECS(Fargate) 자동 배포
Route 53 · ALB · ACM 기반 HTTPS 라우팅
RDS · S3 · CloudWatch · Parameter Store · Secrets Manager 구성



Project.2

HR-Bank

Scheduler로 데이터를 관리하는 Open EMS

02

목차

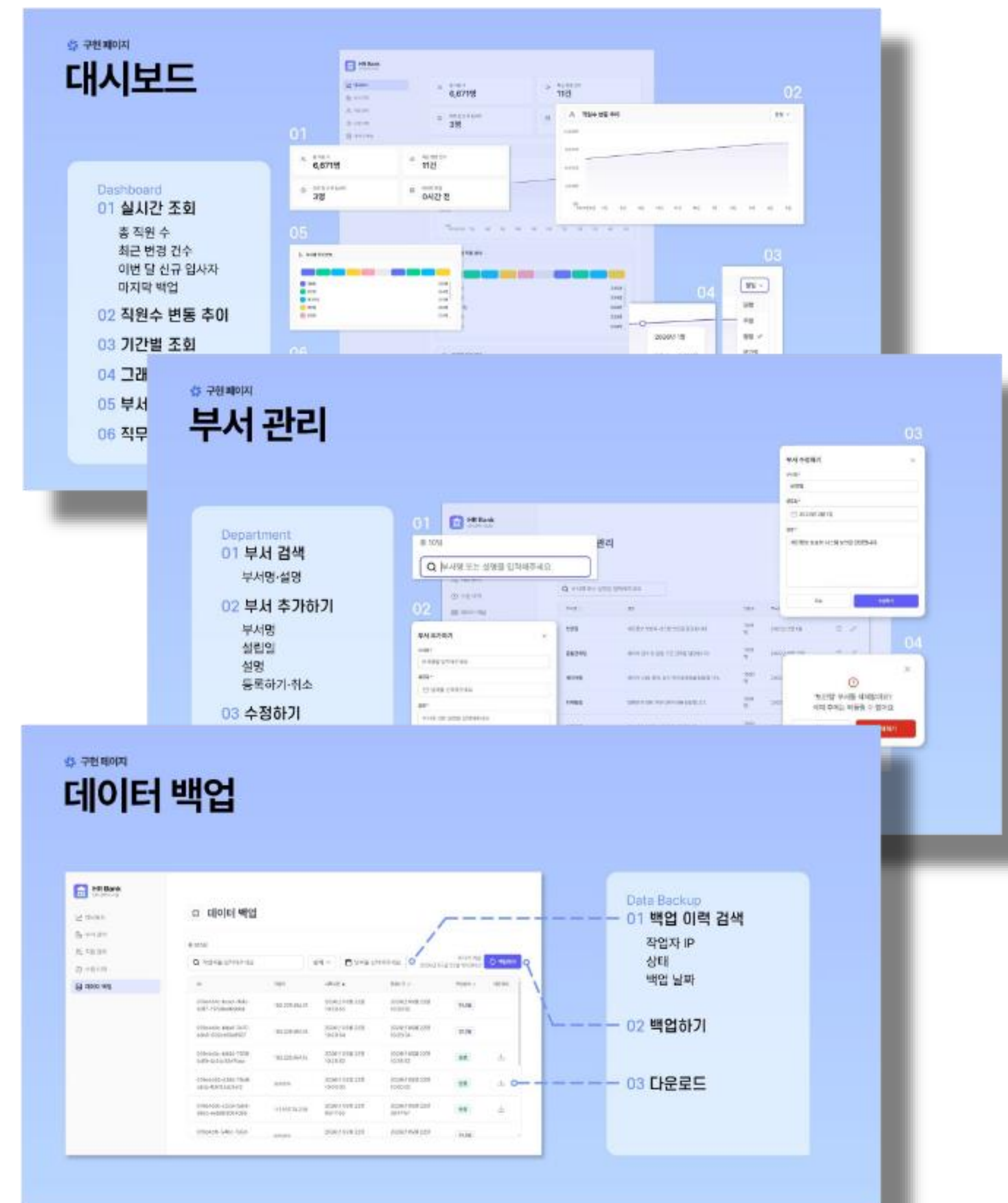
1. 개요
2. 기획 및 설계
3. 기여한 부분 (2)
4. 트러블 슈팅(3)

개요

HR-Bank

배치 기술로 직원·인사 데이터를 자동 관리하는 개방형 HR 플랫폼입니다.

팀장을 맡았으며 프로젝트 관리·기초 구축, 데이터 백업, Spring Scheduler 기반 CSV 자동 백업, Railway 배포를 담당했습니다.



기획 및 설계

①

프로젝트 기획

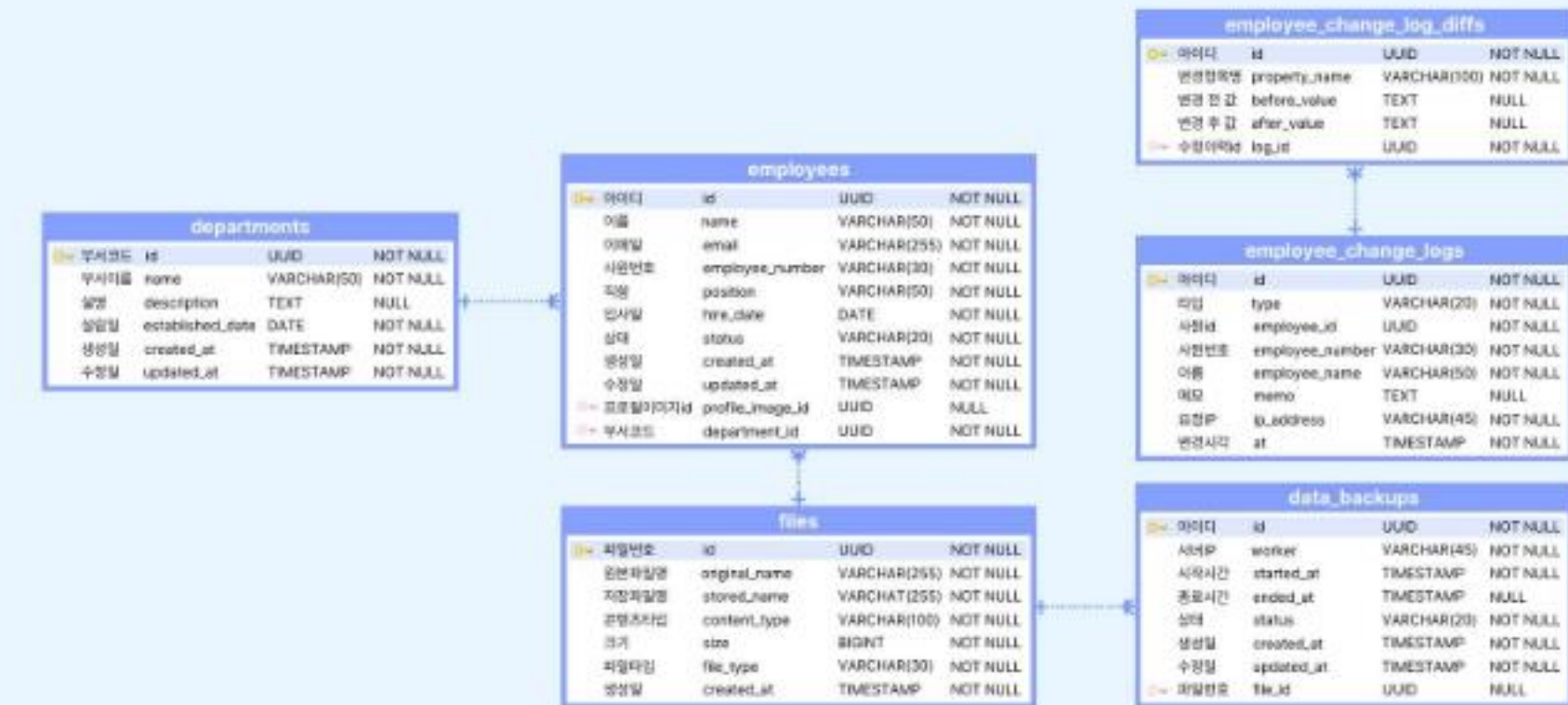
기술 스택 및 개발 환경



②

프로젝트 설계

ERD

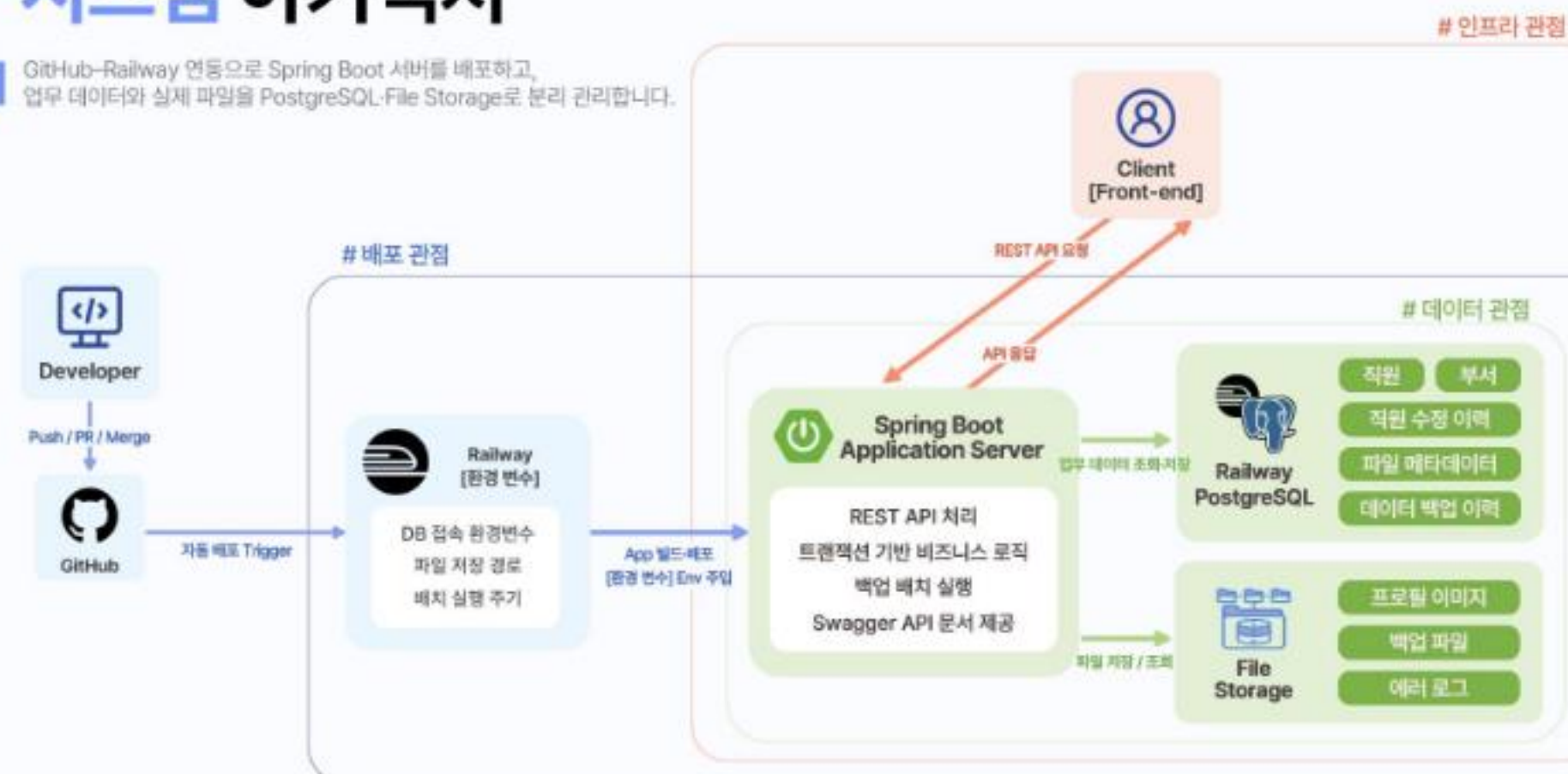


③

프로젝트 설계

시스템 아키텍처

GitHub-Railway 연동으로 Spring Boot 서버를 배포하고, 업무 데이터와 실제 파일을 PostgreSQL-File Storage로 분리 관리합니다.



①

Spring Boot·JPA·QueryDSL 기반으로 기술 스택을 선정하고, Swagger로 API 문서를 자동화했습니다.

②

부서·직원을 중심으로, 변경 이력은 로그와 상세 diff 테이블로 분리하고 파일 메타데이터·백업 이력을 더해 총 6개 테이블로 설계했습니다.

③

GitHub push 시 Railway가 자동 빌드·배포하는 파이프라인을 구성하고, 업무 데이터(PostgreSQL)와 실제 파일(File Storage)을 분리해 관리했습니다.

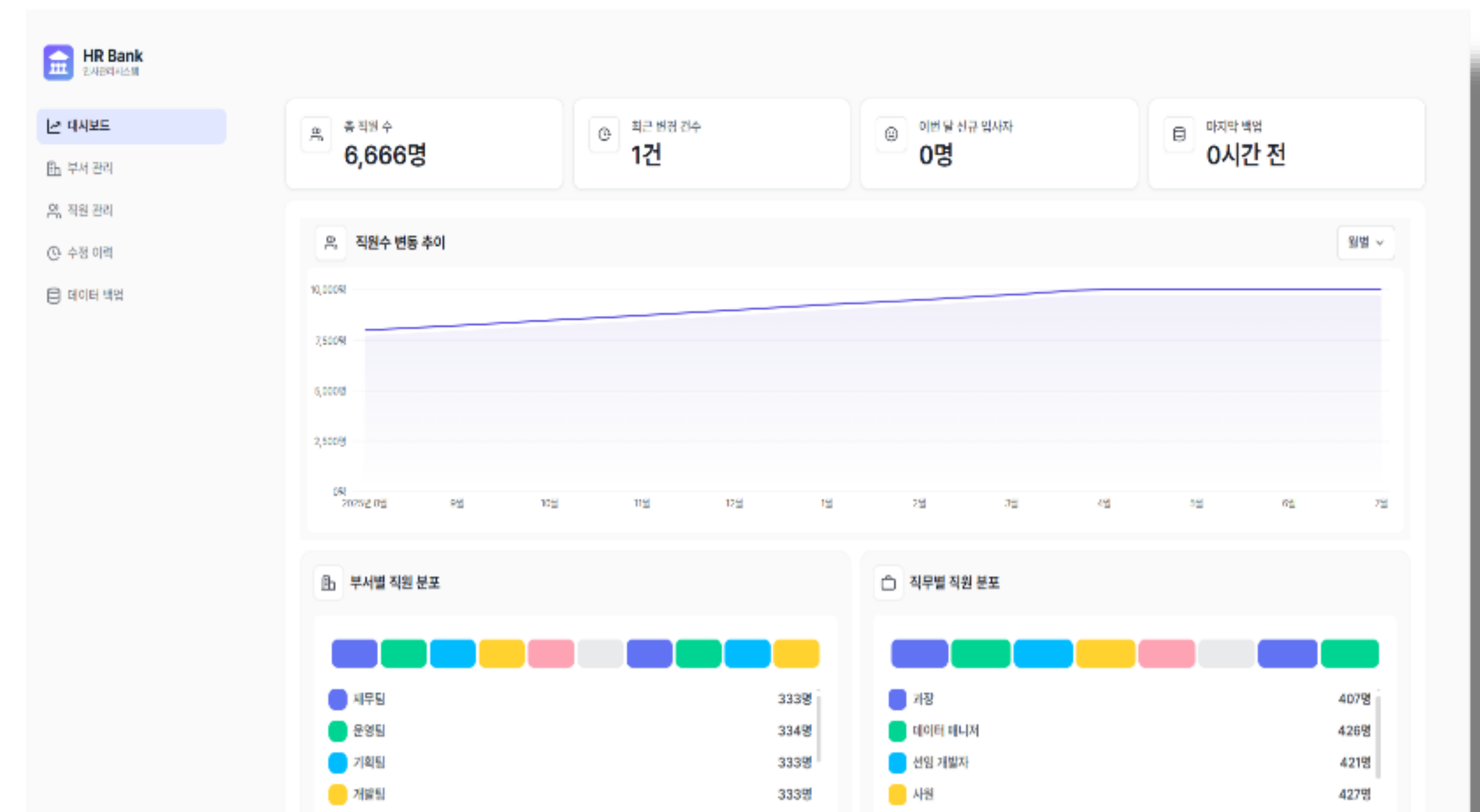
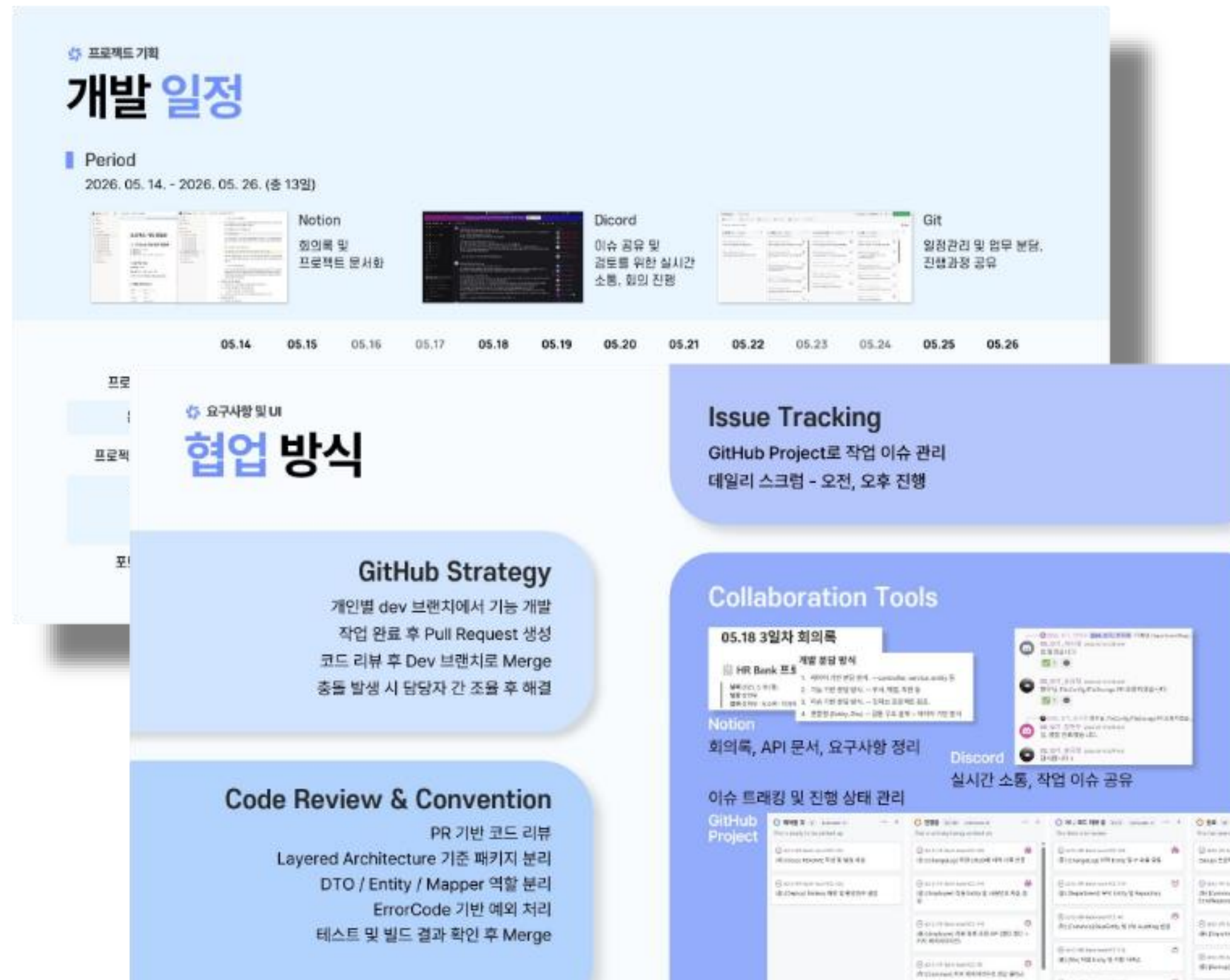
기여한 부분 - 1 / 2

프로젝트 관리·기초 구축

5명 팀원과 일정 조율·작업 분배
공통 응답·전역 예외 처리 구현 및 설정 파일 세팅
Entity(부서, 변경 이력, 직원, 데이터 백업, 파일) 구현

대시 보드 + 동적 검색

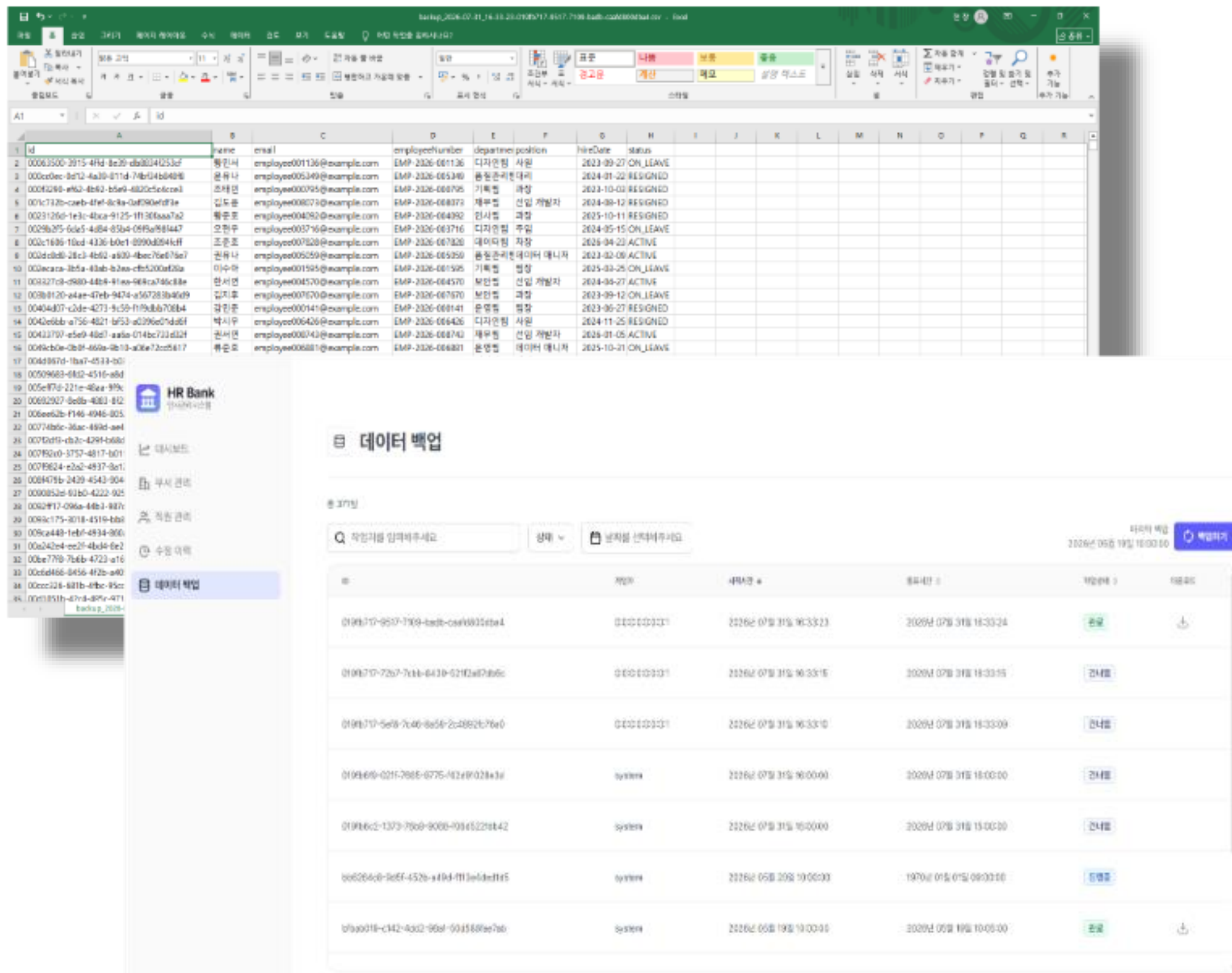
JPA · QueryDSL기반 직원수 · 부서 및 직무별 직원 분포 구현
커서 기반 페이지네이션 구현



기여한 부분 - 2 / 2

데이터 백업·스케줄링

Spring Scheduler 기반 CSV 자동 백업 구현



The image shows a CSV file with employee data and a web application interface for data backup.

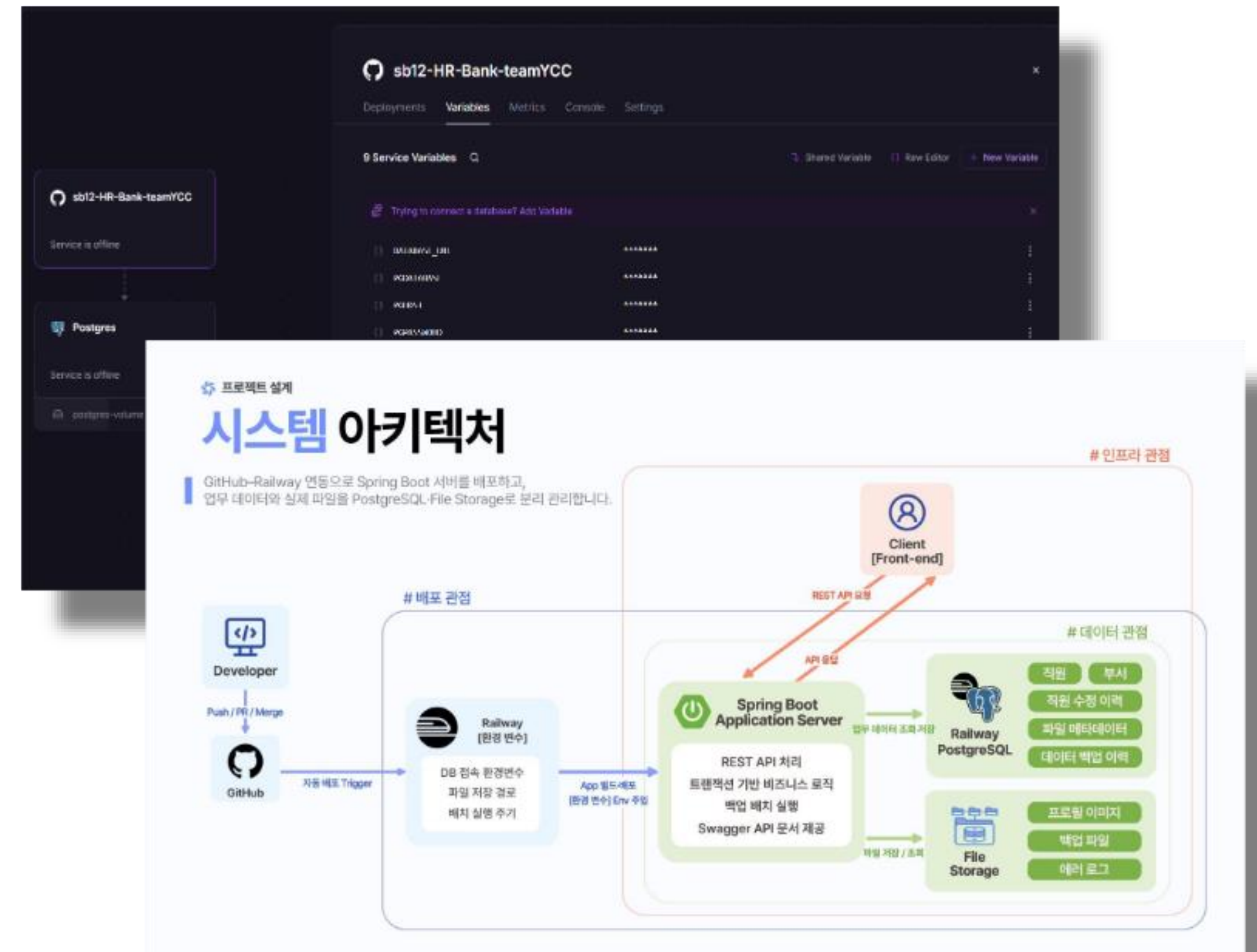
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	id	name	email	employeeNumber	department	position	hireDate	status									
2	00003500-2015-4f1d-8e20-d80240253d	황인서	employee001136@example.com	EMP-2326-001136	디자인팀	사원	2023-09-27	ON_LEAVE									
3	0000c0ec-8e72-4a30-811d-74b04b6496	윤유나	employee005149@example.com	EMP-2326-005149	총괄관리팀	대리	2024-01-22	RESIGNED									
4	000f2666-a762-4b62-b5a9-4820c5c0ca3	조재현	employee000795@example.com	EMP-2326-000795	기획팀	과장	2023-10-03	RESIGNED									
5	001c732b-c9eb-47ef-8c8a-0af90af079e	김도훈	employee0000719@example.com	EMP-2326-0000719	재무팀	신입 개발자	2024-08-12	RESIGNED									
6	00231560-1e3e-40ca-9125-1f136aaa7a2	황종호	employee004352@example.com	EMP-2326-004352	인사팀	과장	2025-10-11	RESIGNED									
7	0029b295-65d5-4484-85a4-095a8a84a7	오현우	employee003766@example.com	EMP-2326-003766	디자인팀	주임	2024-05-15	ON_LEAVE									
8	002c1686-10ed-4336-b0e1-8990a054e4f	조준호	employee001620@example.com	EMP-2326-001620	데이터팀	자장	2026-04-22	ACTIVE									
9	003dc08b-28c3-4b62-a030-4bec76e079e7	권유나	employee005059@example.com	EMP-2326-005059	총괄관리팀	데이터 매니저	2023-02-06	ACTIVE									
10	003ecaca-3b5a-43ab-82ea-cf85200af0a	이수아	employee001595@example.com	EMP-2326-001595	기획팀	협장	2025-03-25	ON_LEAVE									
11	003271c8-9880-4404-81ea-984ca745c88e	한서연	employee004570@example.com	EMP-2326-004570	보안팀	신입 개발자	2024-04-27	ACTIVE									
12	003b8120-a4ae-47eb-9474-a567283b46a9	김지후	employee001070@example.com	EMP-2326-001070	보안팀	과장	2023-09-12	ON_LEAVE									
13	00404807-c0de-4273-9159-f19a8b6700b4	강민준	employee000141@example.com	EMP-2326-000141	운영팀	협장	2023-05-27	RESIGNED									
14	0040e0bb-a756-4021-bf53-a0796e7dabf	박시우	employee006426@example.com	EMP-2326-006426	디자인팀	사원	2024-11-25	RESIGNED									
15	00431707-a5e9-40e7-a5a0-014bc723e024	한서연	employee000743@example.com	EMP-2326-000743	재무팀	신입 개발자	2026-01-05	ACTIVE									
16	004c843a-2b79-460a-8b18-a08e73c0817	황종호	employee006887@example.com	EMP-2326-006887	운영팀	데이터 매니저	2025-10-21	ON_LEAVE									

The web application interface shows a table of data backup records with columns for ID, Name, Email, Employee Number, Department, Position, Hire Date, Status, and Action. The table contains several rows of data, including records for 'HR Bank' and '데이터 백업'.

Railway를 통한 아키텍처 배포

GitHub 연동 자동 배포 구성 — push 시 빌드·배포 실행

dev/prod 프로파일 분리, PostgreSQL·File Storage 운영 환경 설정



트러블 슈팅 - 1 / 3

백업 실패 이력 유실

문제 백업 중 예외가 발생하면 진행·실패 이력까지 함께 롤백되어, 실패 사실 자체가 기록되지 않았습니다.

원인 backup() 전체가 하나의 트랜잭션이라, CSV 쓰기 단계의 예외가 앞선 이력 INSERT까지 되돌렸습니다.

해결 같은 클래스 내부 호출은 프록시를 거치지 않아 어노테이션이 무시되므로, 이력 저장을 별도 빈으로 분리해 트랜잭션 단위로 처리했습니다.

DataBackupService.java

BEFORE

```
@Transactional // 메서드 전체가 하나의 트랜잭션
public DataBackupDto backup(String worker) {
    DataBackup backup = dataBackupRepository.save(
        DataBackup.builder().worker(worker)
            .status(BackupStatus.IN_PROGRESS).build());
    writeCsvInChunks(csvPath); // ← 여기서 예외가 나면
} // 위 INSERT까지 롤백 → 이력 소실
```

DataBackupTxService.java

AFTER

```
public DataBackupDto backup(String worker) { // @Transactional 제거
    DataBackup backup = dataBackupTxService.createInProgressBackup(worker);
    writeCsvInChunks(csvPath);
}

@Transactional
public DataBackup createInProgressBackup(String worker) {
    return dataBackupRepository.save(backup); // IN_PROGRESS 즉시 커밋
}
```

트러블 슈팅 - 2 / 3

데이터 백업 중 OOM

문제 100만건 이상의 대용량 백업 시
OutOfMemoryError문제가 발생했습니다.

원인 전 직원을 한 번에 로딩·누적한 뒤 기록해, 힙 사용량이 데이터량에 비례했습니다.

해결 1,000건 청크 조회와 BufferedWriter 순차 기록(스트리밍) 방식으로 바꿨습니다.

●●● DataBackupService.java

BEFORE

```
List<Employee> all = employeeRepository.findAll(); // 전 직원 메모리 로딩
StringBuilder sb = new StringBuilder();
for (Employee e : all)
    sb.append(toCsvRow(e)).append("\n"); // 계속 누적 → 힙 폭증
Files.writeString(csvPath, sb.toString()); // 대용량 시 OOM
```

●●● DataBackupService.java

AFTER

```
private static final int CHUNK_SIZE = 1000;

private void writeCsvInChunks(Path csvPath) throws IOException {
    try (BufferedWriter writer = Files.newBufferedWriter(csvPath, StandardCharsets.UTF_8)) {
        writer.write('\uFEFF'); // UTF-8 BOM (Excel 한글 깨짐 방지)
        writer.write(CSV_HEADER);
        writer.newLine();

        int page = 0;
        Page<Employee> chunk;
        do {
            chunk = employeeRepository.findAllWithDepartmentForBackup(
                PageRequest.of(page++, CHUNK_SIZE)); // department JOIN FETCH
            for (Employee e : chunk.getContent()) {
                writer.write(toCsvRow(e));
                writer.newLine();
            }
        } while (chunk.hasNext()); // 1,000건씩 스트리밍 → 메모리 일정
    }
}
```

트러블 슈팅 - 3 / 3

직원 목록 조회 N+1

문제

직원 목록을 조회한 뒤 각 직원의 부서명을 매핑할 때, 직원 수만큼 부서 SELECT가 추가로 발생했습니다.

원인

Employee-Department가 지연 로딩(LAZY)이라, 목록의 각 행마다 부서를 개별 조회했습니다.

해결

목록 JPQL에 JOIN FETCH e.department를 적용해 직원과 부서를 한 쿼리로 로딩했습니다.

결과

목록 조회가 1 + N회에서 1회로 줄었고, 페이징 COUNT는 일반 JOIN으로 분리했습니다.

EmployeeRepositoryCustomImpl.java

BEFORE

```
// 목록만 조회 (department 는 LAZY)
List<Employee> list = em.createQuery(
    "SELECT e FROM Employee e ORDER BY e.id", Employee.class)
    .setMaxResults(limit).getResultList();

for (Employee e : list)
    dto.setDepartment(e.getDepartment().getName()); // 1명마다 부서 SELECT → N+1
```

EmployeeRepositoryCustomImpl.java

AFTER

```
// 직원 + 부서를 한 쿼리로 로딩 (N+1 제거)
StringBuilder jpql = new StringBuilder(
    "SELECT e FROM Employee e JOIN FETCH e.department d WHERE 1=1");

appendFilters(jpql, params, ...); // 동적 검색 조건
appendCursorCondition(jpql, params, ...); // 커서 페이징 조건

// 페이징 COUNT 는 FETCH 없이 일반 JOIN 으로 분리
"SELECT COUNT(e) FROM Employee e JOIN e.department d WHERE 1=1";
```

Project.3

유니티 프로젝트

유니티 및 기타프로젝트

03

목차

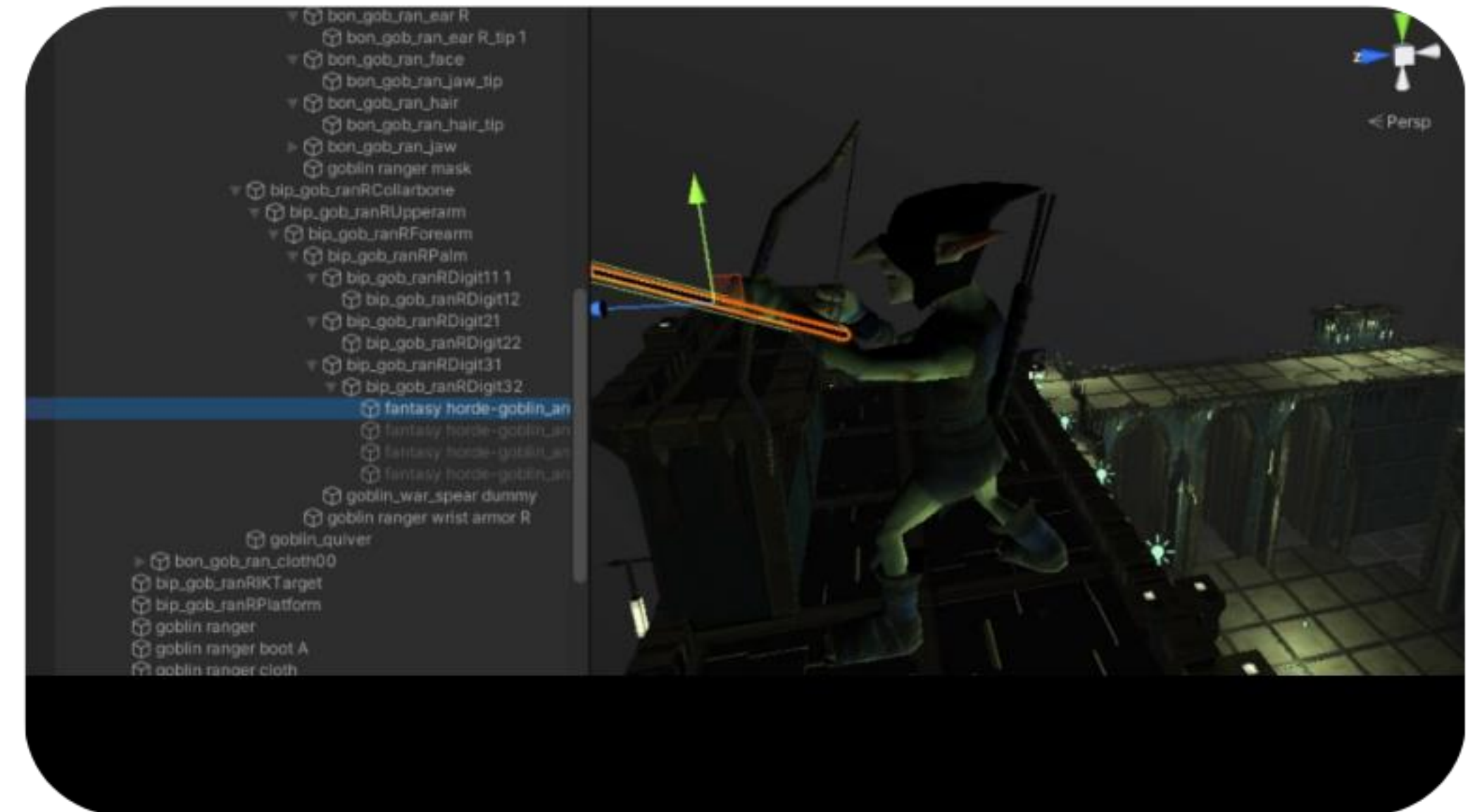
1. Zombie VR
2. 동물의 숲
3. Cub 3D

ZOMBIE VR - VR 사격 게임



주요 특징

- 난이도 선택, 부위별 피격, 슈팅, 몸의 랜덤 스폰 등 다양한 게임 기능 구현
- 모자 착용, 총기 선택 및 조작, 무기 선택, 등산 등 다양한 VR 상호작용 구현



기술적 구현

- 싱글톤, 옵저버 패턴 등 게임 프로그래밍 패턴으로 효율적인 코드 구조 설계
- 오브젝트 풀링을 통한 메모리 효율성 최적화
- 상태 패턴을 활용한 좀비 AI 구현으로 유연한 행동 제어

동물의 숲 - VR 힐링 게임 (팀 프로젝트)



담당 기능

- 인벤토리, 퀘스트, NPC 상호작용, 건축, 농사, 등반 등 다양한 VR 상호작용 구현

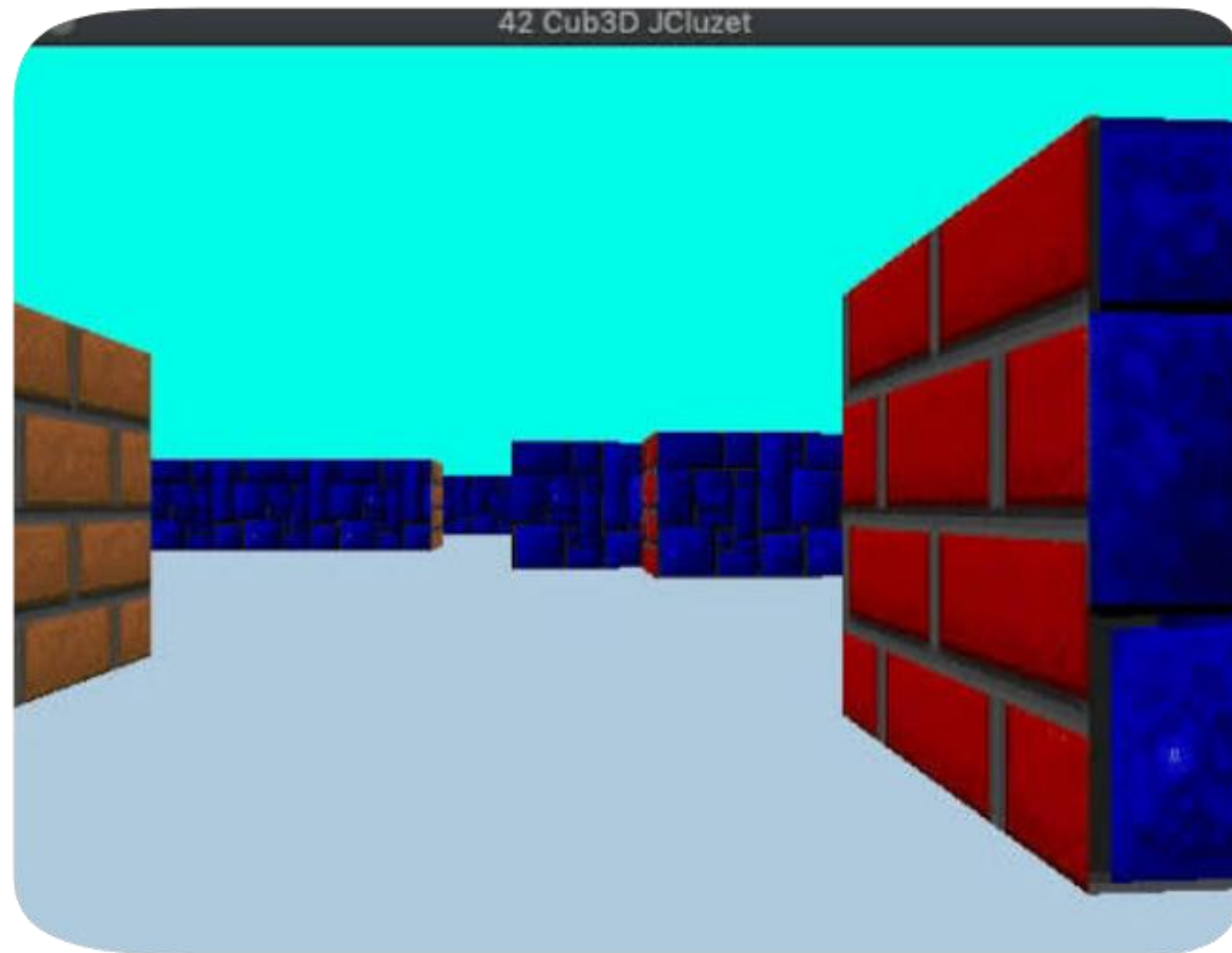


기술적 구현

- XR Interactable 상속으로 상호작용 계층 설계
- 오픈소스 그리드 시스템을 VR 입력에 맞게 이식·확장
- 단일 대화 UI에 NPC 콜백 주입

Cub 3D - Raycasting 알고리즘 기반 3D 게임

42 서울의 대표 그래픽 과제로 Raycasting 알고리즘을 이용한 초기 3D 게임 구현



프로젝트 개요

42 서울의 유명 소프트웨어 그래픽 과제로 울펜슈타인 3D 스타일 게임 구현

구현 원리

화면의 각 수직선마다 벽까지의 거리를 계산하고 원근법을 적용하여 렌더링

핵심 기술

기하학적 거리 계산, 원근법 렌더링, 실시간 성능 최적화 구현